Requirements as the Oracle

Generating REST API Tests from Vague Requirements Without Learning the Bugs They Should Catch

Mid-term progress report

Team PreTrained

Sohail Memon	2025201075	Mohd Ahmad	2025201029
Kathan Patel	2025201039	Md Taufique Hussain	2025202007

Code: `https://github.com/SohailTSM/Requirements-as-the-Oracle`

1 Summary of progress

We are building an LLM agent that writes REST API tests from *vague* natural-language requirements plus an OpenAPI specification, using only Llama 3.1 8B. Two questions drive the work. First, can a small model catch requirement bugs from vague wording? Second, can refinement against a possibly buggy service be made safe, so that the test does not learn the bug it should catch? We evaluate on RESTestBench [1].

- **Built.** The full pipeline runs end to end through RESTestBench's unmodified harness. It comprises retrieval, the precision layer, the planner, lint and specification clean-up, the compiler, execution, the judge and guarded refinement. All 27 component variants of the proposal are implemented, including ReAct, multi-persona, blackboard, MCP and A2A, as are baselines B1–B4 and the fuzzer.
- **Run.** 69 of the 81 planned baseline runs (one run = one service) are complete: every NestJS and TodoApp baseline, the fuzzer, B1 and half of B2/B3 on FastAPI. 24 of the 27 sweep rows are done, with the default configuration repeated 3 times; the remaining 3 rows need the fine-tuned models. Retrieval and the precision layer are evaluated offline. SFT fine-tuning (3 leave-one-service-out folds) is trained, merged and installed.
- **Main result so far.** Refining against the buggy service halves the score of RESTestBench's refinement loop (B2) on FastAPI: 17.0% of planted bugs caught when it refines against the correct service, 8.1% against the buggy one. Our pipeline loses almost nothing (12.0% vs 11.3% for the default; about 17% in both settings for the best variants). So on the realistic setting, where the service may be buggy, our best variants catch about twice as many bugs as B2 (Section 6.1).
- **Main weakness.** About 70% of our tests still fail on the *correct* service, mostly because the precision layer derives checks that contradict the real requirement.

Every design decision and its reason is logged (`Plan/Decision_Log.md`, D-01 to D-112).

2 System

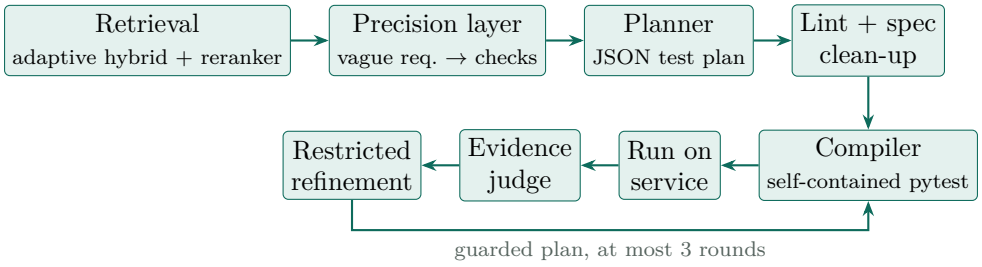


Figure 1: Default pipeline. Everything before the run step uses only the requirement and the specification frozen from the correct service.

The model writes a plan, not code. The LLM writes a structured plan (actors, calls, checks). A deterministic compiler turns the plan into a self-contained pytest file. The plan can then be checked

against the specification before any call, and refinement knows exactly which expected values are locked.

Safety rules for refinement.

(1) On assertion failures, refinement sees status codes and field names but never response values.

(2) A strengthen-only guard restores any change to a meaning check, and blocks new expected values copied from a response.

(3) Field repairs must land on fields the specification documents.

(4) The judge may blame the test only with evidence from the requirement or the specification.

Specification-only clean-up. Before execution, plan errors are fixed deterministically, e.g. missing path parameters are filled in and checks the specification shows can never pass are dropped.

3 Datasets

Dataset	Content and use
RESTestBench [1] (public)	3 Dockerised services: FastAPI (Python), NestJS RealWorld (TypeScript), TodoApp (.NET). 106 requirements (63 / 29 / 14), each in a vague and a precise version; 228 hand-written planted bugs (mutants; 141 / 65 / 22); one reference test per requirement; scoring harness. The agent sees only the vague text and the specification. Precise text is used only for scoring and gold data.
Gold operations (ours)	For each requirement, the specification operations its reference test uses (106 requirements). Ground truth for retrieval.
Gold checks (ours)	268 atomic checks drafted by Claude from the precise requirements. Ground truth for the precision layer. Being validated by human labels (below).
Reference plans (ours)	10 hand-written plans for FastAPI that pass the correct service 10/10 and catch 26 of 28 planted bugs. Used to check that the compiler and runtime are correct.
SFT data (ours)	Vague requirement → verified checks, one fold per held-out service; 86–184 examples per fold.
DPO data (ours)	"Report the bug" (chosen) vs "copy the observed value" (rejected) refinement pairs. Built from guarded runs; collection still pending.
Human-validation set	45 requirements, 181 labels (are the gold checks correct, and do the precision layer's checks cover them), on a shared labelling page. Gives Cohen's κ between labellers and against the Llama judge.

Table 1: Datasets used.

4 Experimental setup

Model and compute. Every LLM role uses Llama 3.1 8B Instruct, 4-bit (Q4_K_M) under Ollama, with 24k context, temperature 0.2 and at most 3 refinement rounds. The baselines are rerun on the same model, so every comparison is like for like.

- Early runs used the team's RTX 5050 laptop. Most runs since 23 Oct use Kaggle T4 servers on the four team accounts, reached through a tunnel. A local request dispatcher sends each LLM call to the next free slot (2 requests per T4 measured at 1.40× throughput).
- The services under test run locally in Docker, with up to 10 FastAPI copies at once so that scoring never leaves a model slot idle.
- The harness is pinned and never edited.
- FastAPI is the development service; NestJS and TodoApp are held out from every design decision.

Metrics.

- *Caught*: percentage of planted bugs caught by tests generated while interacting with the correct service. A test counts only if it passes on the correct service. This is RESTestBench's score.

- *Mutant-gen*: the same score for tests generated while interacting with the buggy service, which is the realistic setting.
- *Adaptation gap*: Caught minus Mutant-gen.
- *Pass-OK*: share of tests that pass on the correct service.
- Component metrics: retrieval Recall@5, "all needed operations in the top 5" and MRR; precision-layer recall, contradiction and unsupported rates against the gold checks.

5 Baseline experiments

B1 Single-step generation, RESTestBench's own tool.

B2 RESTestBench's refinement loop, unchanged.

B3 B2 with only the prompts fixed: the copying example removed, the generator warned, judge feedback passed on, and an API_BUG outcome added.

B4 A response-judging agent in the style of LogiAgent [2].

E is the Schemathesis fuzzer. It uses a crash/schema oracle and no requirements.

Tool	Wording	FastAPI	NestJS	TodoApp	Runs (FastAPI / others)
B1 single-step	precise	39.7 ± 2.6	5.1 ± 1.8	1.5 ± 2.6	3 / 3
B2 refinement	precise	49.6	6.2 ± 2.7	0.0	1 / 3
B3 prompt-fixed	precise	36.2	5.6 ± 4.9	0.0	1 / 3
B4 response judge	precise	*queued*	10.3 ± 5.8	4.5	0 / 3
B1 single-step	vague	4.3 ± 1.4	0.0	0.0	3 / 3
B2 refinement	vague	17.0 ± 3.0	0.0	4.5	2 / 3
B3 prompt-fixed	vague	4.3 ± 4.0	0.0	0.0	2 / 3
B4 response judge	vague	*queued*	3.1 ± 4.1	18.2	0 / 3
E fuzzer	–	13.9 ± 7.8	0.0	9.1	3 / 3

Table 2: Baselines: planted bugs caught (%), mean ± sd over complete runs. Runs still in progress are left out. The remaining FastAPI runs are queued.

Observations.

- **Vague wording collapses single-step generation.** On FastAPI, B1 catches 40% of planted bugs from precise requirements but 4% from vague ones, and 95% of its vague tests fail even on the correct service. This is the gap our method targets.
- **Fixing the prompts alone does not help on vague wording.** B3 scores 4.3% against B2's 17.0%. B3's judge answered API_BUG in 36 of 36 TodoApp rounds, so it almost never refines: the 8B model takes the "it's the API's fault" option whenever it is offered.
- **NestJS and TodoApp are near zero for every LLM baseline.** On TodoApp the model registers users with weak passwords that the service's undocumented password policy rejects; our runtime generates strong credentials for actors. These services are held out and will show whether our pipeline generalises.
- **B2 copies response values.** In 12–22% of its refinement rounds, B2 adds an asserted literal taken from the previous HTTP response. This is exactly the behaviour our guard blocks.

6 Our agent: component sweep (one-at-a-time ablations)

Starting from the default configuration, the sweep changes one component at a time on all 63 FastAPI requirements (vague wording, 1 run each). Removal rows (no precision layer, no refinement, no judge, full specification instead of retrieval, raw pytest instead of plans) act as ablations of each

component. The default ran 3 times to measure noise: 13.5 / 11.3 / 11.3% (mean 12.0, sd 1.3). Differences within about ±2.5 points are therefore treated as noise. The final ablations on all three services follow after the final configuration is chosen (Section 9).

Component	Variant	Caught %	Δ vs default	Mutant-gen %
Default (3 runs)	adaptive retrieval, catalogue precision, plan + compiler, combined refinement, evidence judge	12.0	–	11.3
Retrieval	dense	16.3	**+4.3**	13.6
	query rewriting / BM25 / hybrid	14.9 / 13.5 / 13.5	+2.8 / +1.4 / +1.4	14.1 / 11.5 / 13.6
	full specification (no retrieval)	7.1	**−5.0**	12.4
Precision layer	self-consistency (5 drafts)	17.0	**+5.0**	16.5
	none / direct rewrite	12.8 / 11.3	+0.7 / −0.7	13.9 / 9.6
Generation	raw pytest (no plan, no compiler)	0.0	**−12.1**	–
Refinement	unrestricted / prompt-fixed	15.6 / 15.6	+3.5 / +3.5	12.6 / 12.7
	hidden values only	11.3	−0.7	12.5
	strengthen-only	7.1	**−5.0**	15.5
	none	9.2	−2.8	–
Judge	none	17.0	**+5.0**	10.9
	RESTestBench judge	14.9	+2.8	8.1
Architecture	multi-persona	10.6	−1.4	8.6
	single ReAct agent with tools	5.0	**−7.1**	6.5
	multi-persona + blackboard	3.5	**−8.5**	5.2
Communication	A2A agents / MCP tools	14.9 / 12.1	+2.8 / 0.0	14.4 / 14.3
	content: full trace / verdict only	18.4 / 13.5	**+6.4** / +1.4	15.7 / 12.1
Model	SFT, SFT + DPO, fine-tuned precision layer	*pending: needs a server with the fine-tuned models*		

Table 3: Component sweep, FastAPI, vague wording (63 requirements, 141 planted bugs). Coloured deltas exceed the default's run-to-run spread; single runs, to be confirmed in the finals.

Reading.

- **The structured plan is essential.** With raw pytest generation every test fails on the correct service (0%). The model writes placeholder credentials such as `Bearer YOUR_TOKEN`, which supports H3.
- **Pasting the whole specification hurts** (−5.0), as does handing control to a single ReAct agent or a blackboard (−7 to −8.5). The 8B model often writes tool calls as plain text or stops early.
- **Candidates for the final configuration:** self-consistency in the precision layer, dense retrieval, passing the full trace between agents, and dropping the judge. The judge almost always answers API_BUG (17 of 17 rounds in development runs), so it mostly removes repair chances. These four will be combined and confirmed with 3 runs each.

6.1 Bug adaptation: refining against a buggy service

FastAPI, vague (unless noted)	Caught %	Mutant-gen %	Adaptation gap	Pass-OK %
B2 refinement (2 runs)	17.0	8.1	**8.9**	23.0
B2 refinement, precise (1 run)	49.6	36.6	**13.0**	54.0
B3 prompt-fixed (2 runs)	4.3	5.6	−1.3	4.8
Ours, default (3 runs)	12.0	11.3	0.8	25.9
Ours, precision self-consistency	17.0	**16.5**	0.5	34.9
Ours, content: full trace	18.4	**15.7**	2.7	28.6

Table 4: Adaptation gap: tests generated while interacting with the correct service (Caught) vs with the buggy service (Mutant-gen).

Reading.

- **B2 learns the bugs.** It loses half its score when it refines against the buggy service (17.0 → 8.1), and 13 points on precise wording. This is the bug-adaptation effect RESTestBench reports.
- **Our pipeline does not.** Its score is almost unchanged (gap 0.5–2.7 points), so the best variants reach about twice B2's score on the buggy service. On the correct service, our default is still below B2, and the best variants only match it.
- **Open question.** The adaptation gap stays small for all our refinement variants, including "unrestricted". The protection may therefore come mostly from the plan structure (locked checks, deterministic compiler) rather than from the guard alone. The final ablations test this directly.

7 Component results (offline)

Retrieval (vague)	R@5	All in top 5	MRR
BM25	0.79	0.69	0.70
dense (bge-small)	0.88	0.75	0.89
hybrid	0.85	0.73	0.83
hybrid + reranker	0.89	0.81	0.87
adaptive (default)	**0.89**	**0.81**	0.87
+ query rewriting	0.84	0.76	0.73

Precision layer	Recall	Contr.	Unsup.
catalogue, vague	0.39	0.27	0.06
direct rewrite, vague	0.40	0.21	0.09
self-consist., vague	0.37	0.26	0.07
catalogue, precise	0.47	0.14	0.10
direct rewrite, precise	0.54	0.08	0.07

Table 5: Left: retrieval over 106 requirements (operations other than login/registration). Right: precision layer vs gold checks, scored by a Llama judge (to be validated by human labels).

Retrieval. The reranker helps short vague requirements but hurts long precise ones (precise R@5: 0.70 with the reranker vs 0.90 hybrid). The default therefore reranks only requirements of at most 30 words ("adaptive").

Precision layer. It recovers about 40% of the gold checks from vague wording. About a quarter of its checks contradict the gold, and these are the main source of tests that fail on the correct service. Refinement may not remove them without risking adaptation, which makes the precision layer, and its fine-tuning, the component with the most room to improve.

Fine-tuning (SFT). QLoRA (rank 16) on Kaggle T4s takes 20–41 minutes per fold, with final losses of 0.21–0.35. Our Ollama version no longer serves LoRA adapters, so each fold is merged and exported as a 4-bit GGUF model on Kaggle, then installed locally. The three SFT models are ready for the sweep's model rows.

8 Challenges and mitigations

Challenge	Mitigation
Compute. The remaining work is about 380 run-hours, too much for one 8 GB laptop GPU, which also overheated.	Inference moved to Kaggle T4 servers on all four accounts, behind one local dispatcher (2 requests per GPU, 1.40×); up to 10 FastAPI copies so that scoring never idles a slot. The last batch kept the 8 slots 86% busy, with 13,158 requests and none failed.
12-hour Kaggle sessions cut long runs (B2 on FastAPI needs about 10 h).	Runs are split into parts sized to the time left in the session; cut-off parts are kept aside and rerun, never averaged in.
The harness silently skipped NestJS in early runs: its single health check fails while the service restarts, and the run still reported success.	A longer health check in our launcher; such runs now count as failures; all affected runs were repeated.
Large run-to-run noise. The same logic scored 21–64% on a 5-requirement smoke test.	The sweep uses all 63 requirements, the default runs 3 times, and differences are judged against that spread; finals use 3 runs and paired tests.
The 8B judge always blames the API (ours 17/17 rounds, B3 36/36).	The sweep tests "no judge"; it scores +5 points, so the final configuration will likely drop or change the judge.
About 70% of tests fail on the correct service, mostly from contradicting derived checks.	Self-consistency (+5 points), the fine-tuned precision layer, and human labels to separate judge errors from real contradictions.
ReAct agent: the 8B model wrote tool calls as plain text with invented names.	Text tool calls are unwrapped and executed; the fallback rate is reported with every ReAct run.
LoRA adapters not loadable in Ollama; the Kaggle disk was too small for the merge.	Merge and 4-bit export on Kaggle, with the conversion moved to `/tmp`.
DPO data and fold leakage: guarded runs exist only on FastAPI, so the FastAPI-held-out fold would get no pairs.	Short data-generation runs on NestJS/TodoApp, used only in folds that hold out a different service.

Table 6: Main challenges so far.

9 Timeline to the final submission

Dates	Work	Compute
30 Oct – 2 Nov	Remaining FastAPI baselines (B2/B3 repeats, B4); DPO data runs; server notebook with the fine-tuned models; DPO training (3 folds)	∼1 Kaggle batch + 3 h training
3 – 4 Nov	Sweep rows SFT, SFT+DPO and fine-tuned precision layer; choose the final configuration and 2 runner-ups	∼10 GPU-h
5 – 9 Nov	Our finals: 3 configurations × 2 wordings × 3 runs on all services. Ablations of the final configuration (8 runs). 4-bit vs fp16 check	∼155 GPU-h, 3–4 batches
by 11 Nov	Reruns of failed runs; human labels ($\geq$100, Cohen's κ). **Experiment freeze Wed 11 Nov, 23:59**	–
12 – 15 Nov	Statistics (paired Wilcoxon per requirement, effect sizes), failure taxonomy, final report and slides. **Final submission Sun 15 Nov**	–

Table 7: Plan to the final submission. One batch = two 12-hour Kaggle sessions with 2 T4s each.

About 215 run-hours remain, roughly 4–5 batches. The four team accounts provide about 30 GPU-hours each per week, which is enough before the freeze. If time runs short, the fp16 comparison is dropped first, then the second runner-up.

References

[1] L. Kogler, S. Hangler, M. Ehrhart, B. Dornauer, R. Wuersching, P. Schrammel. *RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements.* EASE 2026. arXiv:2604.25862.

[2] *LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents.* arXiv:2503.15079, 2025.